

WEEK-7

CODE:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #define MAX_TREE_HT 100
4  struct MinHeapNode {
5      char data;
6      unsigned freq;
7      struct MinHeapNode *left, *right;
8  };
9  struct MinHeap {
10     unsigned size;
11     unsigned capacity;
12     struct MinHeapNode** array;
13 };
14 struct MinHeapNode* newNode(char data, unsigned freq) {
15     struct MinHeapNode* temp =
16         (struct MinHeapNode*)malloc(sizeof(struct MinHeapNode));
17     temp->left = temp->right = NULL;
18     temp->data = data;
19     temp->freq = freq;
20     return temp;
21 }
22 struct MinHeap* createMinHeap(unsigned capacity) {
23     struct MinHeap* minHeap =
24         (struct MinHeap*)malloc(sizeof(struct MinHeap));
25     minHeap->size = 0;
26     minHeap->capacity = capacity;
27     minHeap->array =
28         (struct MinHeapNode**)malloc(capacity * sizeof(struct MinHeapNode));
29     return minHeap;
30 }
31 void swapMinHeapNode(struct MinHeapNode** a,
32                      struct MinHeapNode** b) {
33     struct MinHeapNode* t = *a;
34     *a = *b;
35     *b = t;
36 }
37 void minHeapify(struct MinHeap* minHeap, int idx) {
38     int smallest = idx;
39     int left = 2 * idx + 1;
40     int right = 2 * idx + 2;
41
42     if (left < minHeap->size &&
43         minHeap->array[left]->freq <
44         minHeap->array[smallest]->freq)
```

```

45         smallest = left;
46
47         if (right < minHeap->size &&
48             minHeap->array[right]->freq <
49             minHeap->array[smallest]->freq)
50             smallest = right;
51
52     if (smallest != idx) {
53         swapMinHeapNode(&minHeap->array[smallest],
54                         &minHeap->array[idx]);
55         minHeapify(minHeap, smallest);
56     }
57 }
58 struct MinHeapNode* extractMin(struct MinHeap* minHeap) {
59     struct MinHeapNode* temp = minHeap->array[0];
60     minHeap->array[0] =
61         minHeap->array[minHeap->size - 1];
62     --minHeap->size;
63     minHeapify(minHeap, 0);
64     return temp;
65 }
66 void insertMinHeap(struct MinHeap* minHeap,
67                   struct MinHeapNode* minHeapNode) {
68     ++minHeap->size;
69     int i = minHeap->size - 1;
70
71     while (i && minHeapNode->freq <
72            minHeap->array[(i - 1) / 2]->freq) {
73         minHeap->array[i] =
74             minHeap->array[(i - 1) / 2];
75         i = (i - 1) / 2;
76     }
77
78     minHeap->array[i] = minHeapNode;
79 }
80 struct MinHeapNode* buildHuffmanTree(char data[],
81                                     int freq[],
82                                     int size) {
83     struct MinHeapNode *left, *right, *top;
84     struct MinHeap* minHeap = createMinHeap(size);
85
86     for (int i = 0; i < size; ++i)

```

```

87     minHeap->array[i] = newNode(data[i], freq[i]);
88
89     minHeap->size = size;
90
91     while (minHeap->size != 1) {
92         left = extractMin(minHeap);
93         right = extractMin(minHeap);
94
95         top = newNode('$', left->freq + right->freq);
96         top->left = left;
97         top->right = right;
98
99         insertMinHeap(minHeap, top);
100     }
101
102     return extractMin(minHeap);
103 }
104 void printCodes(struct MinHeapNode* root,
105                int arr[], int top) {
106     if (root->left) {
107         arr[top] = 0;
108         printCodes(root->left, arr, top + 1);
109     }
110
111     if (root->right) {
112         arr[top] = 1;
113         printCodes(root->right, arr, top + 1);
114     }
115
116     if (!root->left && !root->right) {
117         printf("%c: ", root->data);
118         for (int i = 0; i < top; ++i)
119             printf("%d", arr[i]);
120         printf("\n");
121     }
122
123 int main() {
124     char arr[] = {'a', 'b', 'c', 'd', 'e', 'f'};
125     int freq[] = {5, 9, 12, 13, 16, 45};
126
127     int size = sizeof(arr) / sizeof(arr[0]);
128

```

```
129     struct MinHeapNode* root =
130         buildHuffmanTree(arr, freq, size);
131
132     int code[MAX_TREE_HT], top = 0;
133
134     printf("Huffman Codes:\n");
135     printCodes(root, code, top);
136
137     return 0;
138 }
```

OUTPUT:

Huffman Codes:

f: 0

c: 100

d: 101

a: 1100

b: 1101

e: 111

Process exited after 0.08794 seconds with return value 0

Press any key to continue . . . |